

Lecture 5

Linear Regression (1)

Getting started with linear regression

Keji He @ SDU-AI

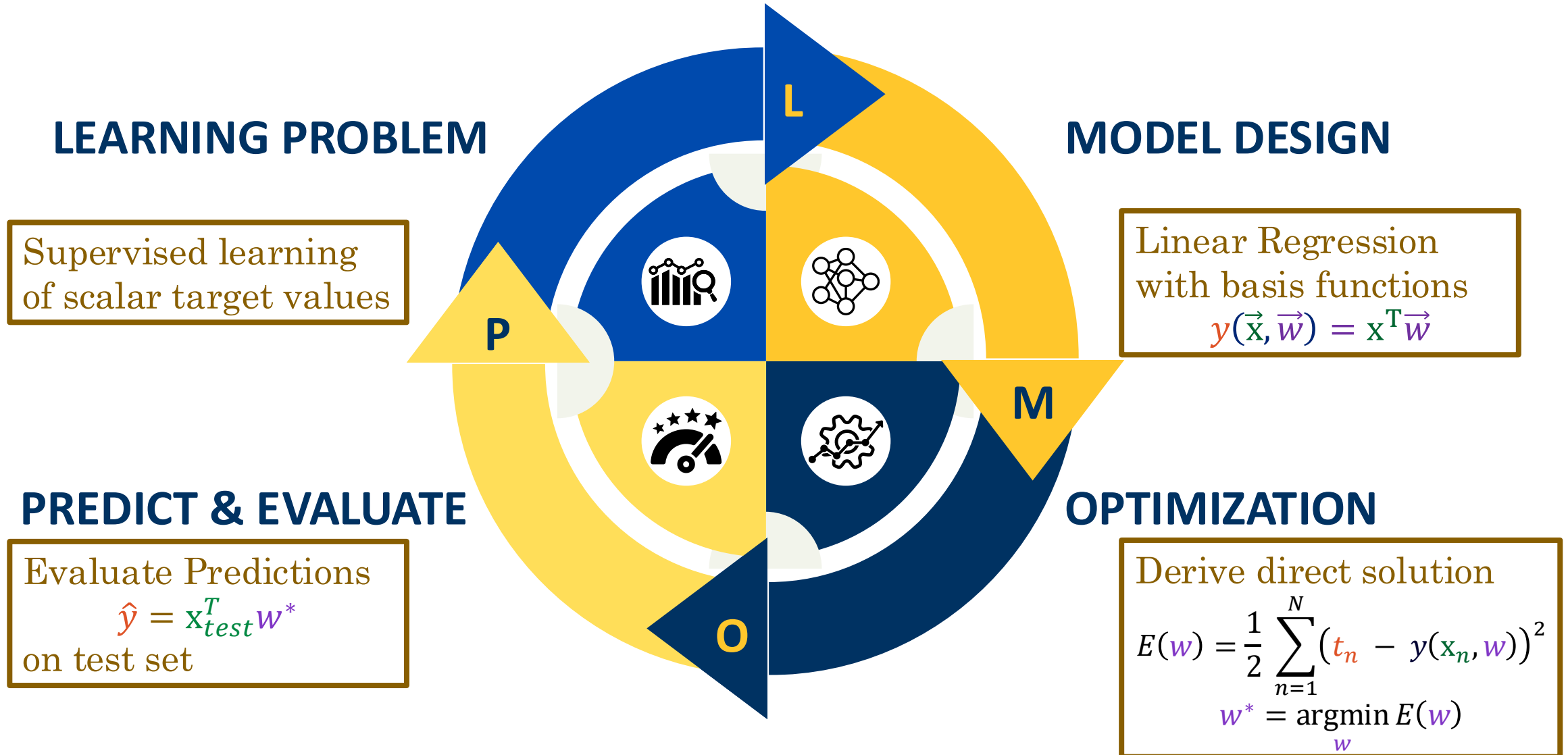
Roadmap

- Linear Regression
- Basis Functions
- Vectorizing Calculations
- Error Function
- Error Function Minimization
- Geometric Interpretation
- Evaluation

Linear Regression

- Linear Regression
- Basis Functions
- Vectorizing Calculations
- Error Function
- Error Function Minimization
- Geometric Interpretation
- Evaluation

Linear Regression Outline



Learning Problem

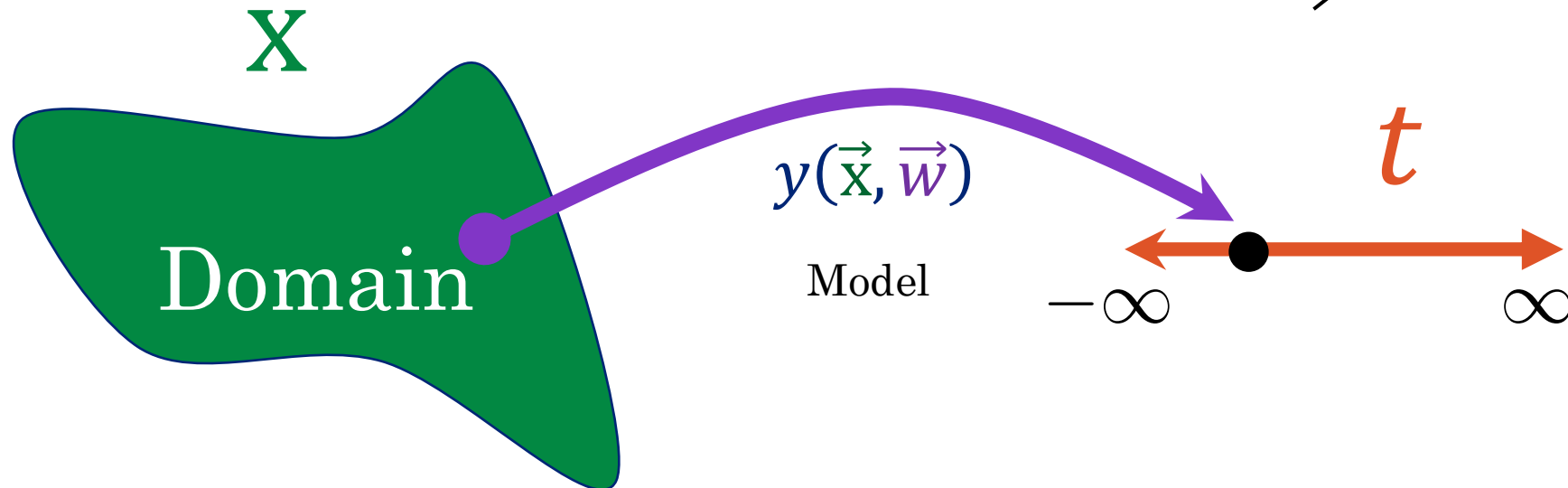
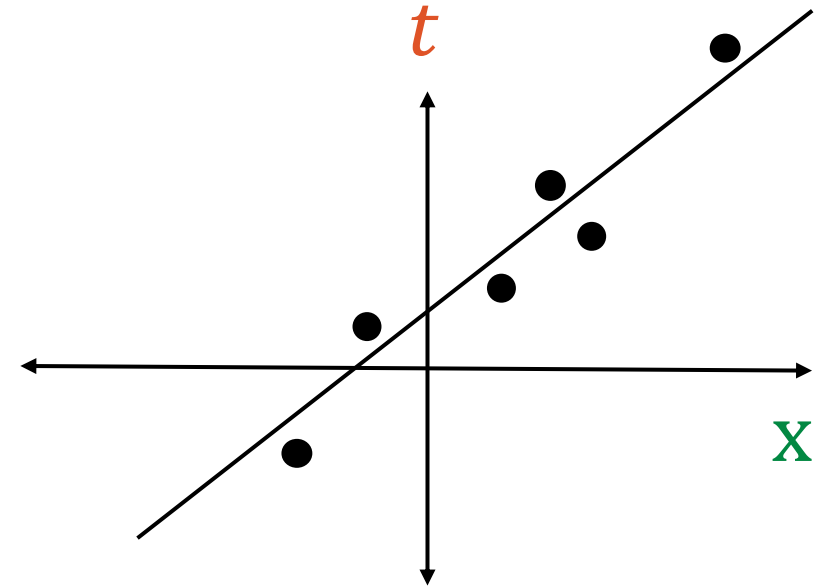
LEARNING PROBLEM

Supervised learning
of scalar target values



Regression

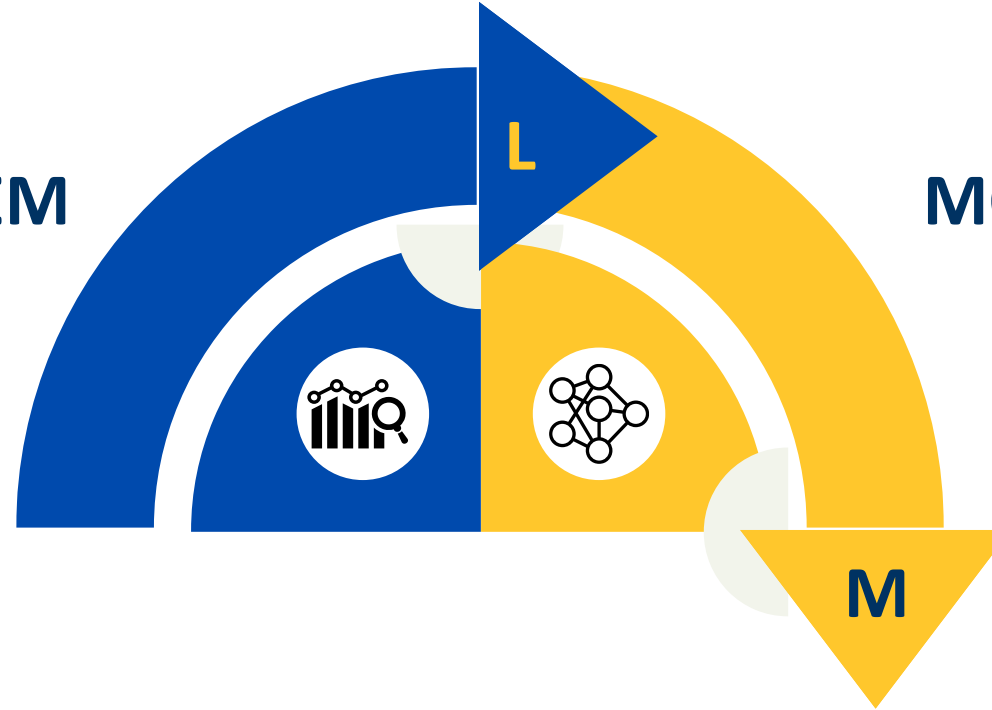
- Estimating relationship between $\mathbf{x}$ and t .
 - t is a quantitative value
 - We will soon see $\mathbf{x}$ can be almost anything ...



Model Design

LEARNING PROBLEM

Supervised learning
of scalar target values



MODEL DESIGN

Linear Regression
with basis functions
 $y(\vec{x}, \vec{w}) = \vec{x}^T \vec{w}$

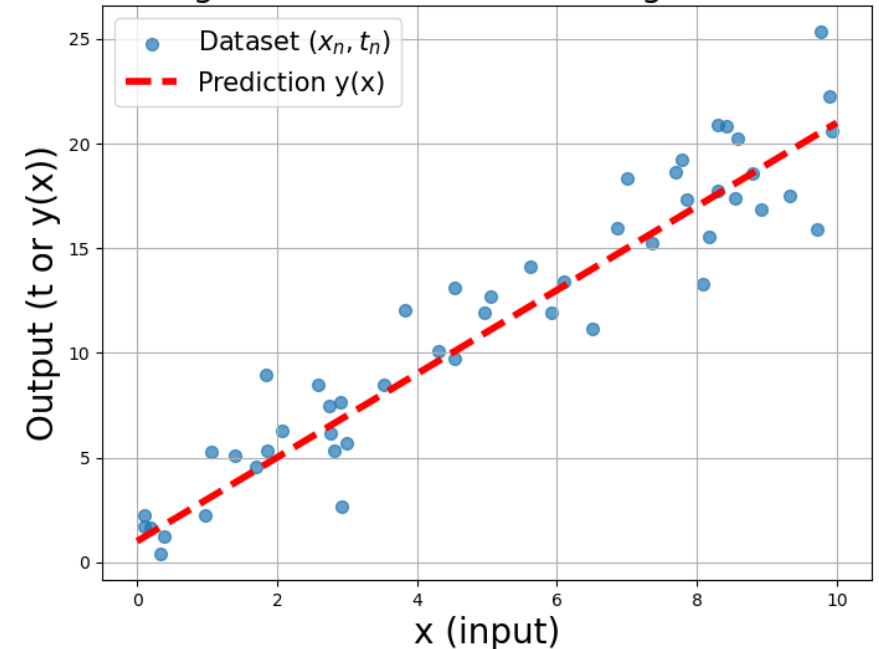
Supervised Linear Regression

- Goal: Predict one or more continuous *target* variable t given the vector $\vec{x} \in \mathbb{R}^D$ of *input* variables.

$$\{(x_n)\}_{n=1}^N \xrightarrow{\vec{w}} \{(t_n)\}_{n=1}^N$$

- We formulate a function $y(\vec{x}, \vec{w})$ whose values for new input $\vec{x}$ constitute the predictions for the corresponding values of t .
- $\vec{w}$ is a vector of parameters that can be learned from the training data.

Visualizing Dataset and a Linear Regression Function



The Simplest Linear Regression Model

- The simplest model is of the following form:

$$y(\vec{x}, \vec{w}) = w_0 + w_1 x_1 + \dots + w_D x_D \quad \text{where } \vec{x} = (x_1, x_2, \dots, x_D)^T \in \mathbb{R}^D.$$

$y(\vec{x}, \vec{w})$ → Predicted output

$w_0 + w_1 x_1 + \dots + w_D x_D$ → Linear combination of parameters $w_0, w_1, \dots, w_D$, each scaled by x_d 's.

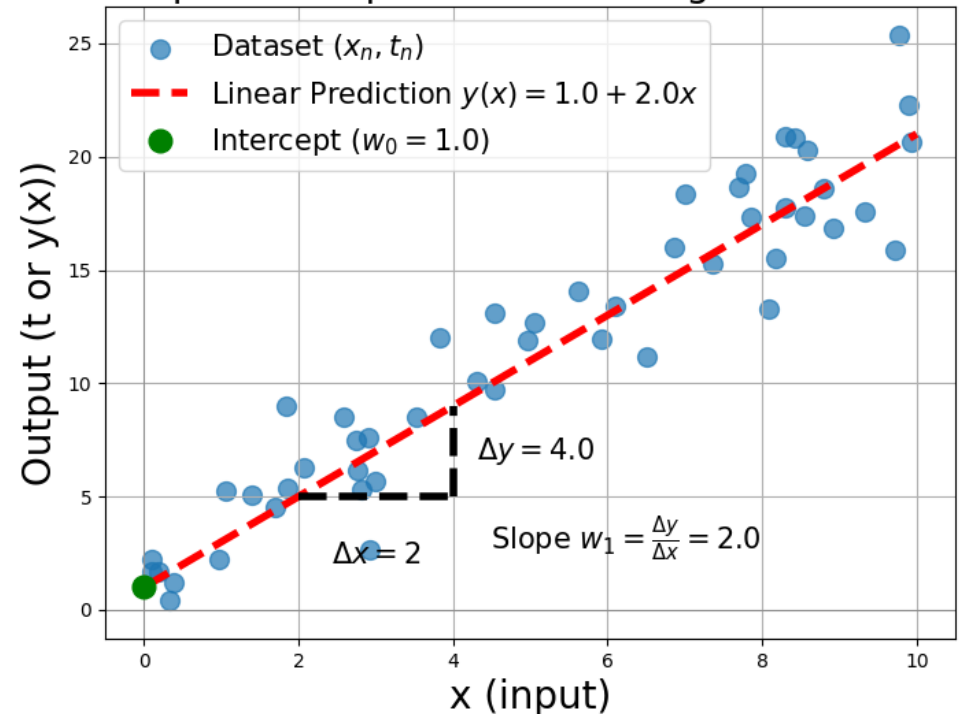
- In the simplest case of $D = 1$, we have:

$$y(\vec{x}, \vec{w}) = w_0 + w_1 x_1$$

w_0 → Intercept (shift)

w_1 → Slope (rate of change)

Intercept and Slope in a Linear Regression Function



The Simplest Linear Regression Model

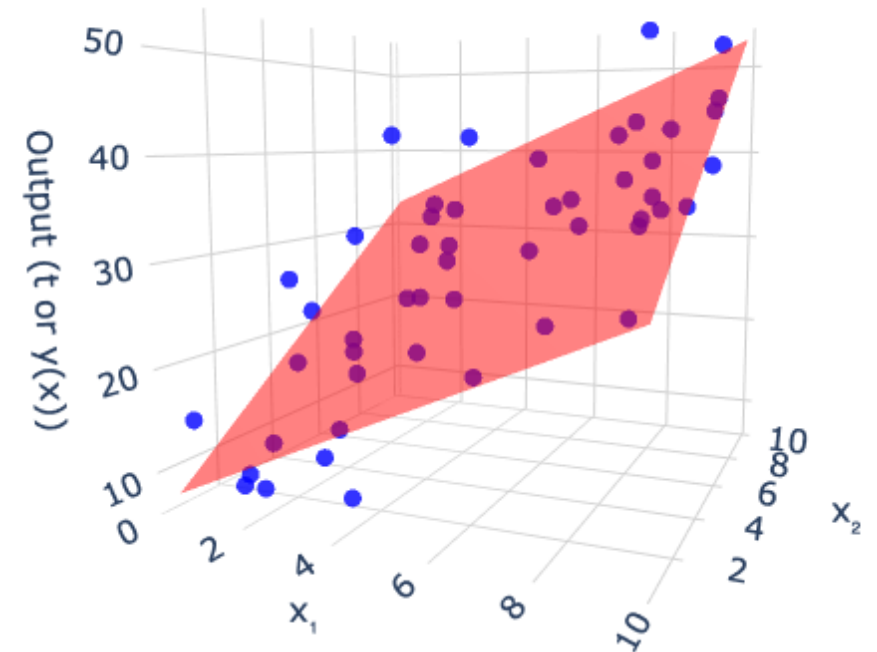
- The simplest model is of the following form:

$$y(\vec{x}, \vec{w}) = w_0 + w_1 x_1 + \dots + w_D x_D \quad \text{where } \vec{x} = (x_1, x_2, \dots, x_D)^T \in \mathbb{R}^D.$$

$y(\vec{x}, \vec{w})$ ↓ Predicted output

$w_0 + w_1 x_1 + \dots + w_D x_D$ Linear combination of parameters $w_0, w_1, \dots, w_D$, each scaled by x_d 's.

- In higher dimension, the predicted values form a hyperplane in $\mathbb{R}^{D+1}$.



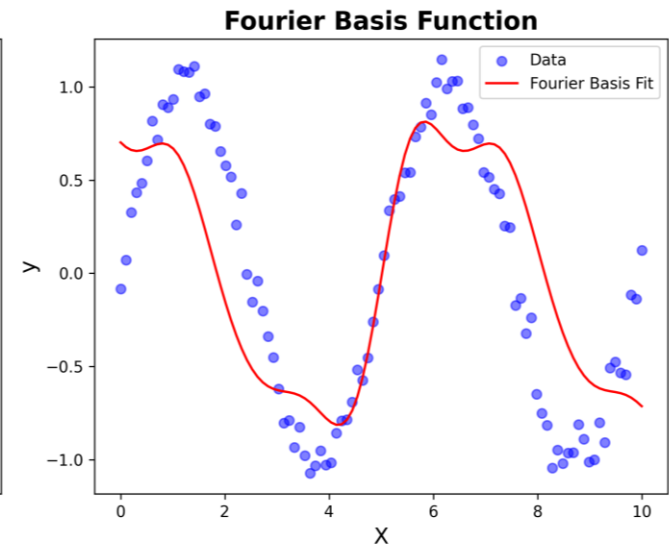
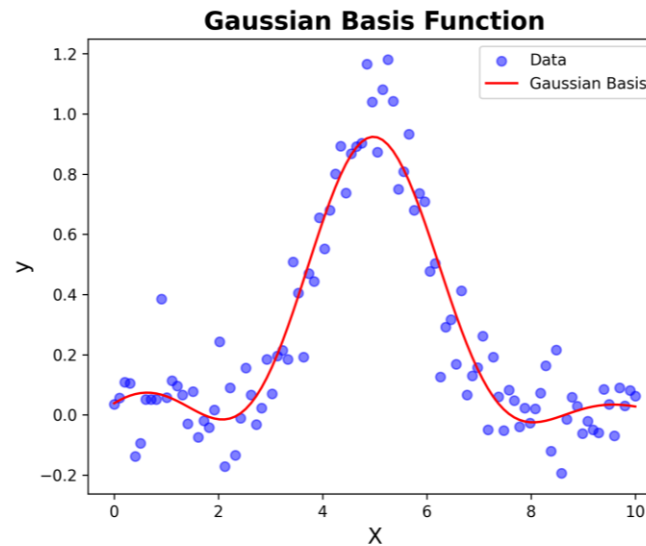
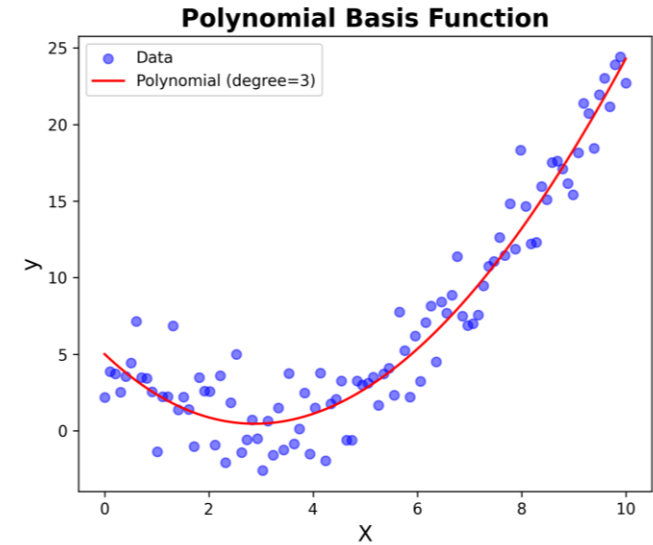
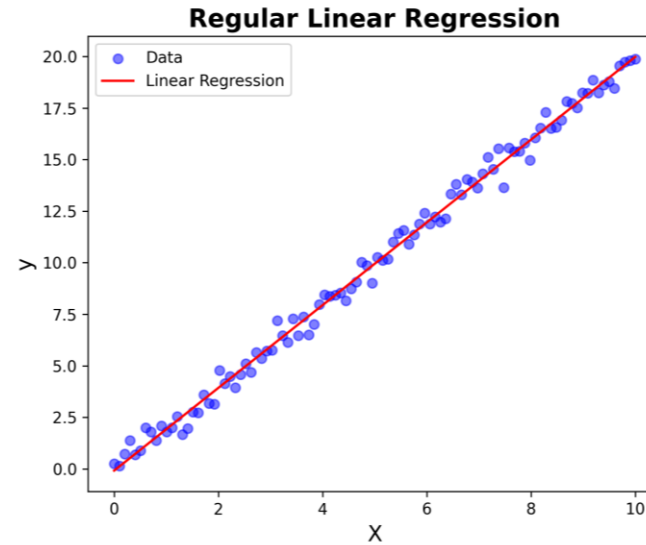
[Interactive link](#)

Basis Functions

- Linear Regression
- Basis Functions
- Vectorizing Calculations
- Error Function
- Error Function Minimization
- Geometric Interpretation
- Evaluation

Linear Functions

Which of the following is a linear regression model?

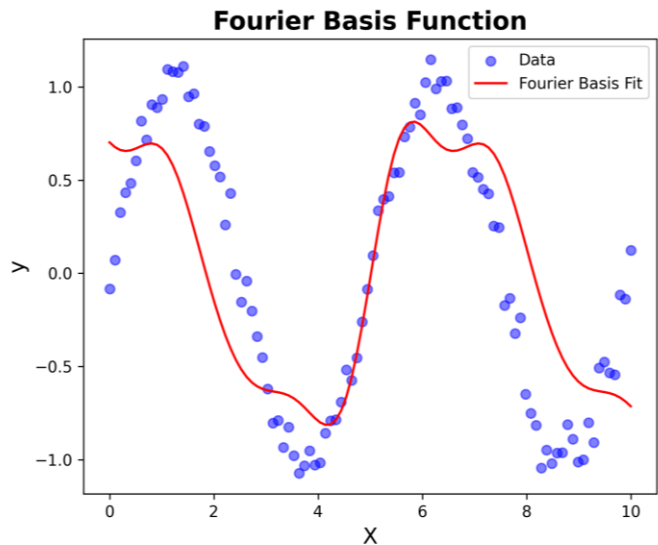
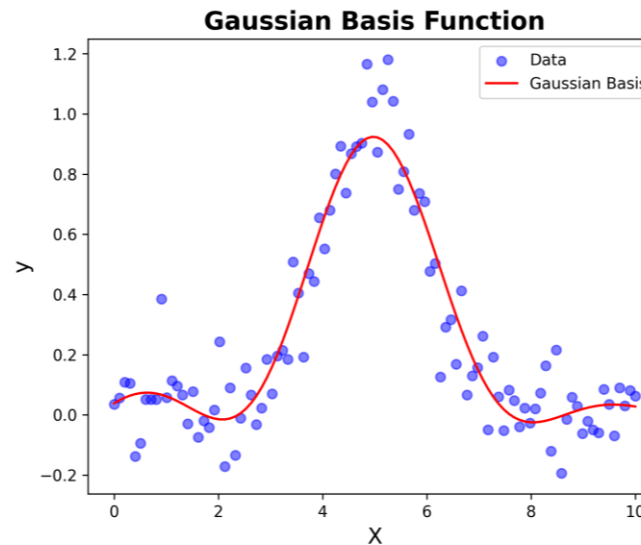
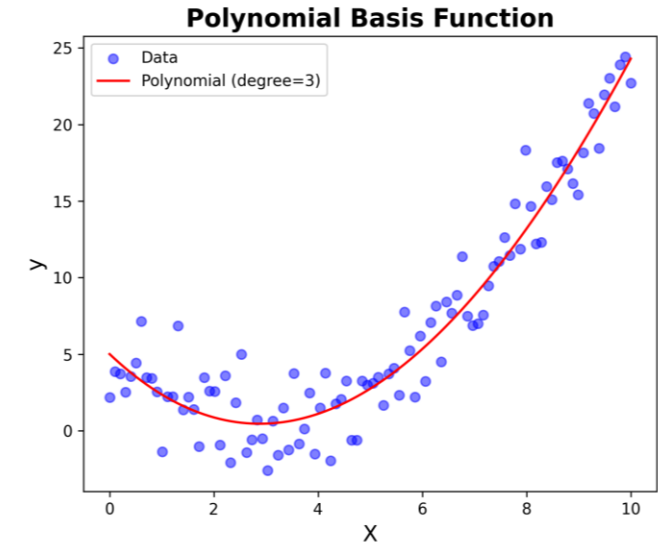
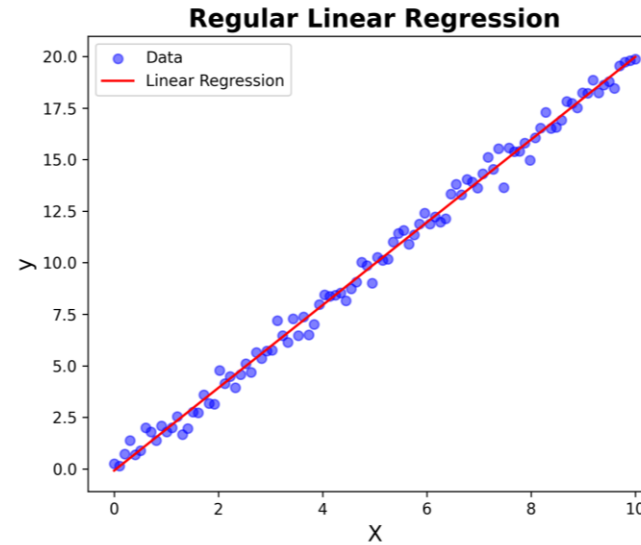


Linear Functions

Which of the following is a linear regression model?

These are all linear models with different basis functions.

We will now see what basis functions are...



What Does It Mean To Be a Linear Model?

In what sense are the previous plots linearly modeled?

$$y(\vec{x}, \vec{w}) = w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\mathbf{x})$$

Are linear models linear in the

1. Features?
2. Parameters?

What Does It Mean To Be a Linear Model?

In what sense are the previous plots linearly modeled?

$$y(\vec{x}, \vec{w}) = w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\vec{x})$$

Linear in the Parameters

Feature Functions

Are linear models linear in the

1. Features?
2. Parameters?

Basis Functions

- We can extend the class of models by considering linear combinations of fixed nonlinear functions of the input variables, of the form:

$$y(\vec{x}, \vec{w}) = w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\vec{x})$$

where $\phi_j(\vec{x})$ are known as basis functions.

More on Basis Function

- **Polynomial basis** (多项式基函数): $\phi_j(x) = x^j$
- **Radial Basis Functions** (RBF, 径向基函数)
 - such as **Gaussian**: $\phi_j(x) = e^{-\frac{(x-\mu_j)^2}{2\sigma^2}}$
- **Sigmoidal basis** (S型基函数): $\phi_j(x) = \text{sig}(\frac{x-\mu_j}{\sigma})$ where $\text{sig}(a) = \frac{1}{1+e^{-a}}$
 - 双曲正切函数 $\tanh(a) = 2\text{sig}(2a) - 1$
- **Sinusoidal basis** (正弦型基函数): $\phi_j(x) = \sin x_j$ or $\phi_j(x) = \cos x_j$

Basis Functions as Features

Why did we care about basis functions before deep learning?

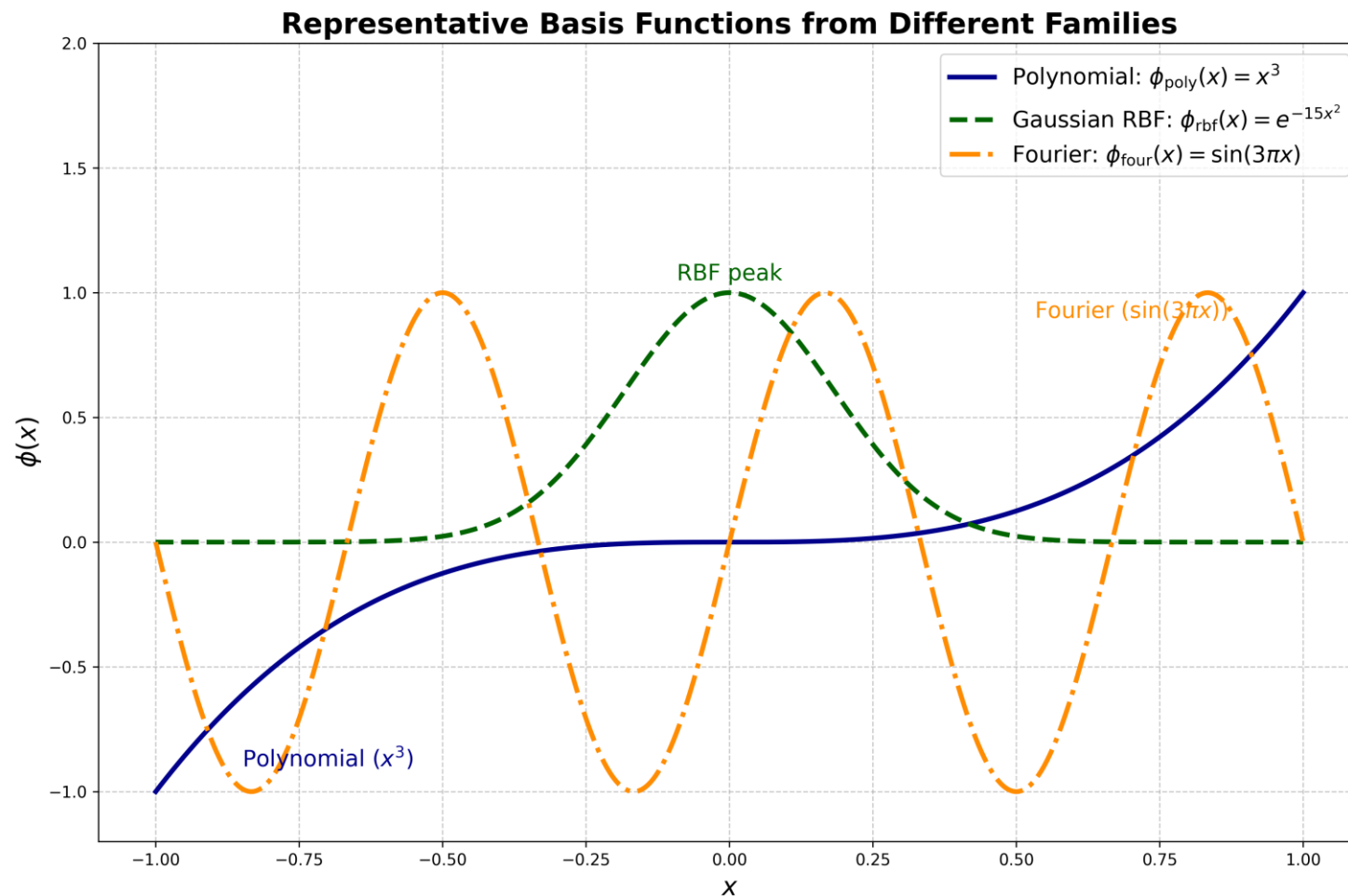
1. Background: Fixed feature extraction was the norm before deep learning:

raw input $x \rightarrow$ hand-designed features $\phi_j(x)$.

2. Reason: Pick expressive features so a simple linear model could succeed.

3. Popular choices: polynomials, radial-basis functions, Fourier terms, etc.

Heavy manual engineering.

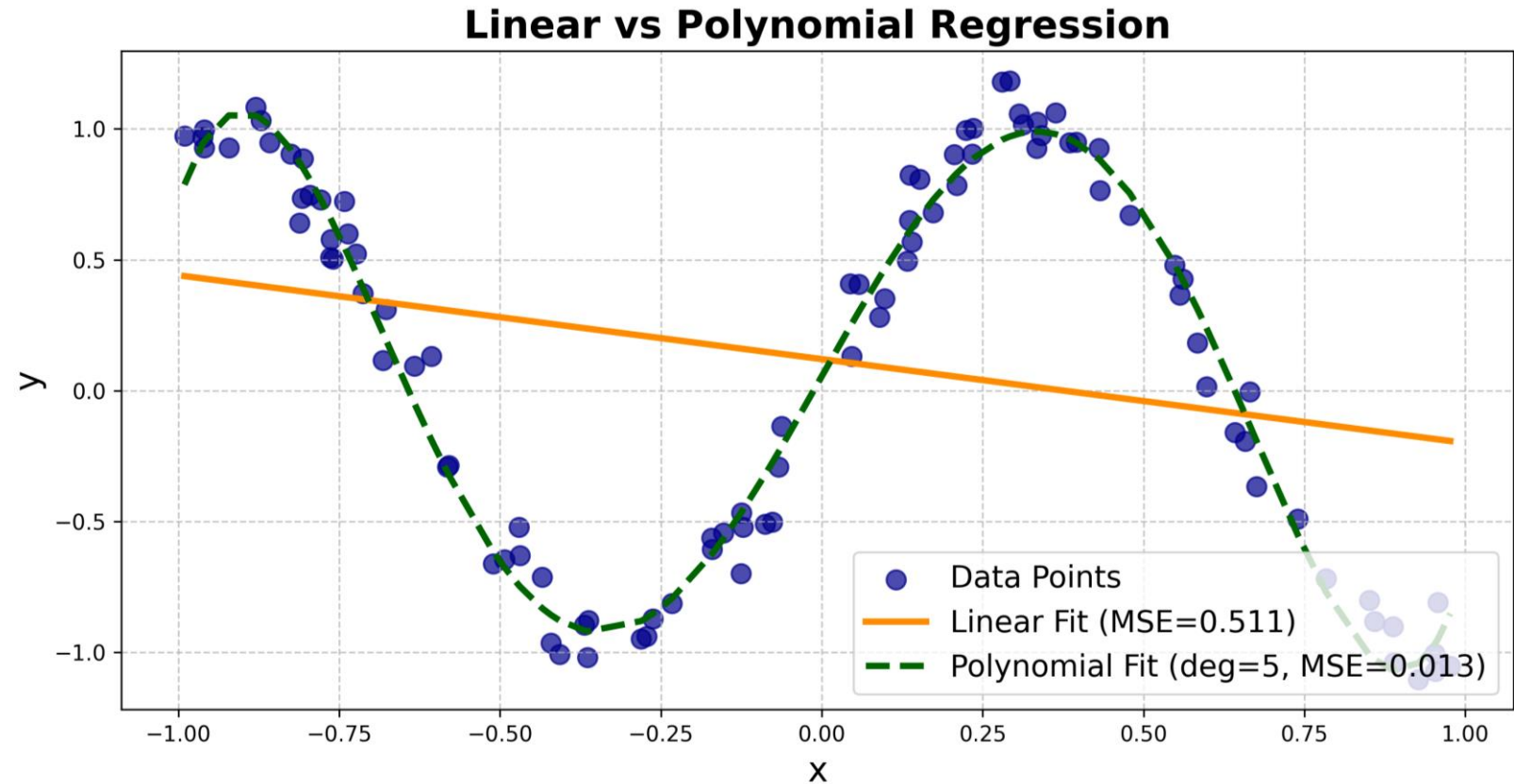


How Would Basis Functions Improve Predictions?

Using polynomial basis functions of degree 5, we redefined the linear regression equation as:

$$y(\vec{x}, \vec{w}) = \sum_{j=0}^5 w_j x^j$$

Looking at the Mean Squared Error (MSE) between targets and predictions, the polynomial fit has a better performance.



$$y(x) = 0.000 + 4.355x^1 - 0.203x^2 - 14.426x^3 + 0.164x^4 + 9.368x^5$$

$$y(x) = 0.120 - 0.320x$$

Vectorizing Calculations

- Linear Regression
- Basis Functions
- Vectorizing Calculations
- Error Function
- Error Function Minimization
- Geometric Interpretation
- Evaluation

Vectorizing Calculations

- Rewriting the equation for linear regression:

$$y(\vec{x}, \vec{w}) = w_0 + w_1 x_1 + \dots + w_D x_D = w_0 + \sum_{d=1}^D w_d x_d$$

- If we assume that there is an $x_0 = 1$, then the two terms above can be combined into one summation:

$$y(\vec{x}, \vec{w}) = w_0 + w_1 x_1 + \dots + w_D x_D = \sum_{d=0}^D w_d x_d$$

where $\vec{x} = (x_0, x_1, x_2, \dots, x_D)^T$ and $\vec{w} = (w_0, w_1, w_2, \dots, w_D)^T$

↓
Augmented input with x_0

↘
 $x_0 = 1$

Vectorizing Calculations

$$y(\vec{x}, \vec{w}) = w_0 + w_1 x_1 + \dots + w_D x_D = \sum_{d=0}^D w_d x_d$$

where $\vec{x} = (x_0, x_1, x_2, \dots, x_D)^T$ and $\vec{w} = (w_0, w_1, w_2, \dots, w_D)^T$

$$\begin{aligned} y(\vec{x}, \vec{w}) &= \vec{w} \cdot \vec{x} \\ &= \vec{w}^T \vec{x} = \vec{x}^T \vec{w} \end{aligned}$$

Matrix Notation

- Assuming we have $n = 1, \dots, N$ samples in our dataset, we can write:


$$\left\{ \begin{array}{l} y(\vec{x}_1, \vec{w}) = w_0 + w_1 x_{11} + w_2 x_{12} + \dots + w_D x_{1D} \\ y(\vec{x}_2, \vec{w}) = w_0 + w_1 x_{21} + w_2 x_{22} + \dots + w_D x_{2D} \\ \dots \\ y(\vec{x}_N, \vec{w}) = w_0 + w_1 x_{N1} + w_2 x_{N2} + \dots + w_D x_{ND} \end{array} \right.$$

$$\left\{ \begin{array}{ll} y(\vec{x}_1, \vec{w}) = \vec{x}_1^T \vec{w} & \text{where } \vec{x}_1^T = [1, x_{11}, x_{12}, \dots, x_{1D}] \\ y(\vec{x}_2, \vec{w}) = \vec{x}_2^T \vec{w} & \text{where } \vec{x}_2^T = [1, x_{21}, x_{22}, \dots, x_{2D}] \\ \dots & \dots \\ y(\vec{x}_N, \vec{w}) = \vec{x}_N^T \vec{w} & \text{where } \vec{x}_N^T = [1, x_{N1}, x_{N2}, \dots, x_{ND}] \end{array} \right.$$

Matrix Notation

$$\begin{cases} y(\vec{x}_1, \vec{w}) = \vec{x}_1^T \vec{w} & \text{where } \vec{x}_1^T = [1, x_{11}, x_{12}, \dots, x_{1D}] \\ y(\vec{x}_2, \vec{w}) = \vec{x}_2^T \vec{w} & \text{where } \vec{x}_2^T = [1, x_{21}, x_{22}, \dots, x_{2D}] \\ \dots & \\ y(\vec{x}_N, \vec{w}) = \vec{x}_N^T \vec{w} & \text{where } \vec{x}_N^T = [1, x_{N1}, x_{N2}, \dots, x_{ND}] \end{cases}$$

↓

$$\begin{cases} y(\vec{x}_1, \vec{w}) = [1, x_{11}, x_{12}, \dots, x_{1D}] \vec{w} \\ y(\vec{x}_2, \vec{w}) = [1, x_{21}, x_{22}, \dots, x_{2D}] \vec{w} \\ \dots \\ y(\vec{x}_N, \vec{w}) = [1, x_{N1}, x_{N2}, \dots, x_{ND}] \vec{w} \end{cases}$$

↓

$$\mathbb{X} = \{\vec{x}_1, \vec{x}_2, \dots, \vec{x}_N\} \in \mathbb{R}^{N \times (D+1)}$$

Design matrix

$\mathbb{Y} = \{y(\vec{x}_1, \vec{w}), y(\vec{x}_2, \vec{w}), \dots, y(\vec{x}_N, \vec{w})\} \in \mathbb{R}^N$

$\in \mathbb{R}^{D+1}$

$\mathbb{Y}(\mathbb{X}, \vec{w}) = \mathbb{X} \vec{w}$

$\in \mathbb{R}^{N \times (D+1)}$

Error Function

- Linear Regression
- Basis Functions
- Vectorizing Calculations
- Error Function
- Error Function Minimization
- Geometric Interpretation
- Evaluation

Optimization

LEARNING PROBLEM

Supervised learning
of scalar target values



MODEL DESIGN

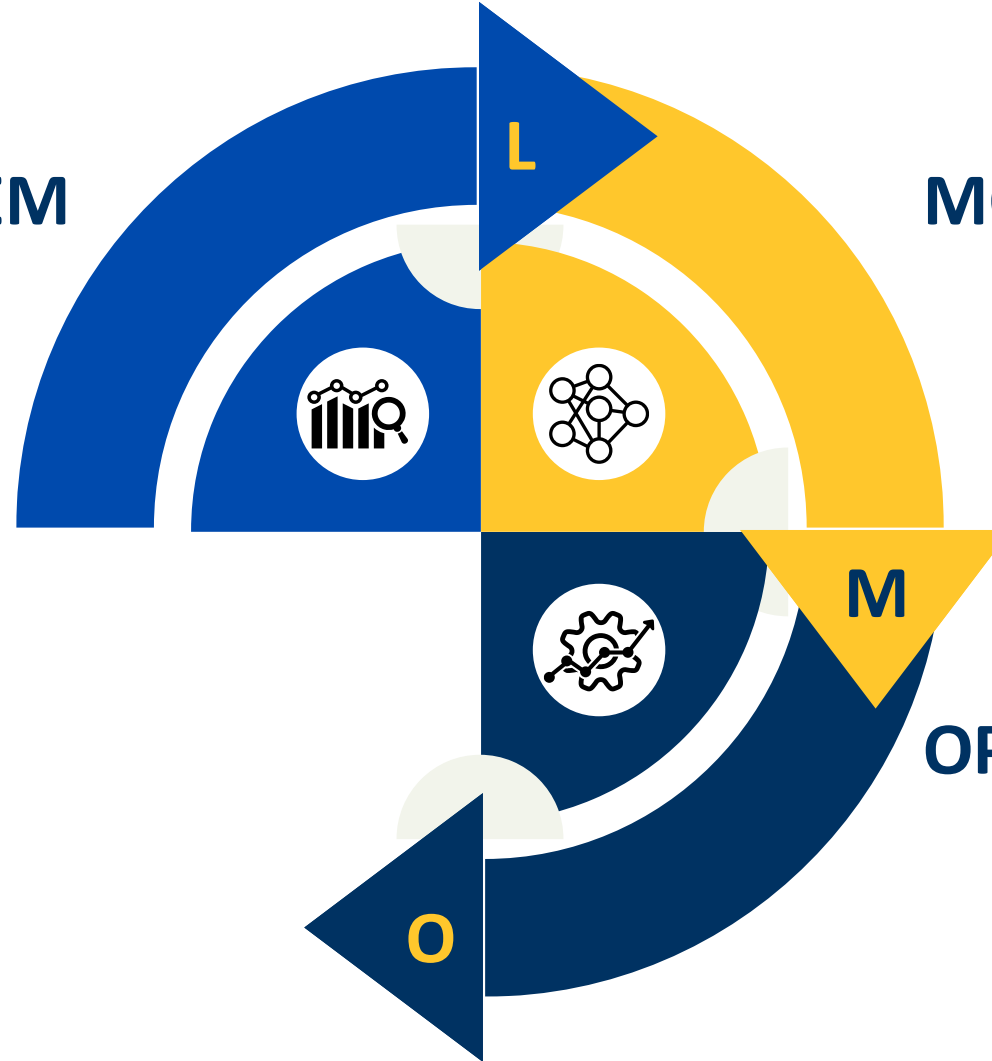
Linear Regression
with basis functions
 $y(\vec{x}, \vec{w}) = \vec{x}^T \vec{w}$



OPTIMIZATION

Derive direct solution

$$E(w) = \frac{1}{2} \sum_{n=1}^N (t_n - y(\mathbf{x}_n, w))^2$$
$$w^* = \underset{w}{\operatorname{argmin}} E(w)$$



Error Function

- How to calculate the weights $\vec{w}$?
 - We do this by minimizing an *error function* that measures the **misfit** between the function $\mathbb{Y}(\mathbb{X}, \vec{w})$ and **targets**, for any given value of $\vec{w}$, and the training data $\mathbb{X}$.
- One simple error function: **Sum of the squares of the differences between targets and the predictions.**

$$E(w) = \frac{1}{2} \sum_{n=1}^N (t_n - y(x_n, w))^2$$

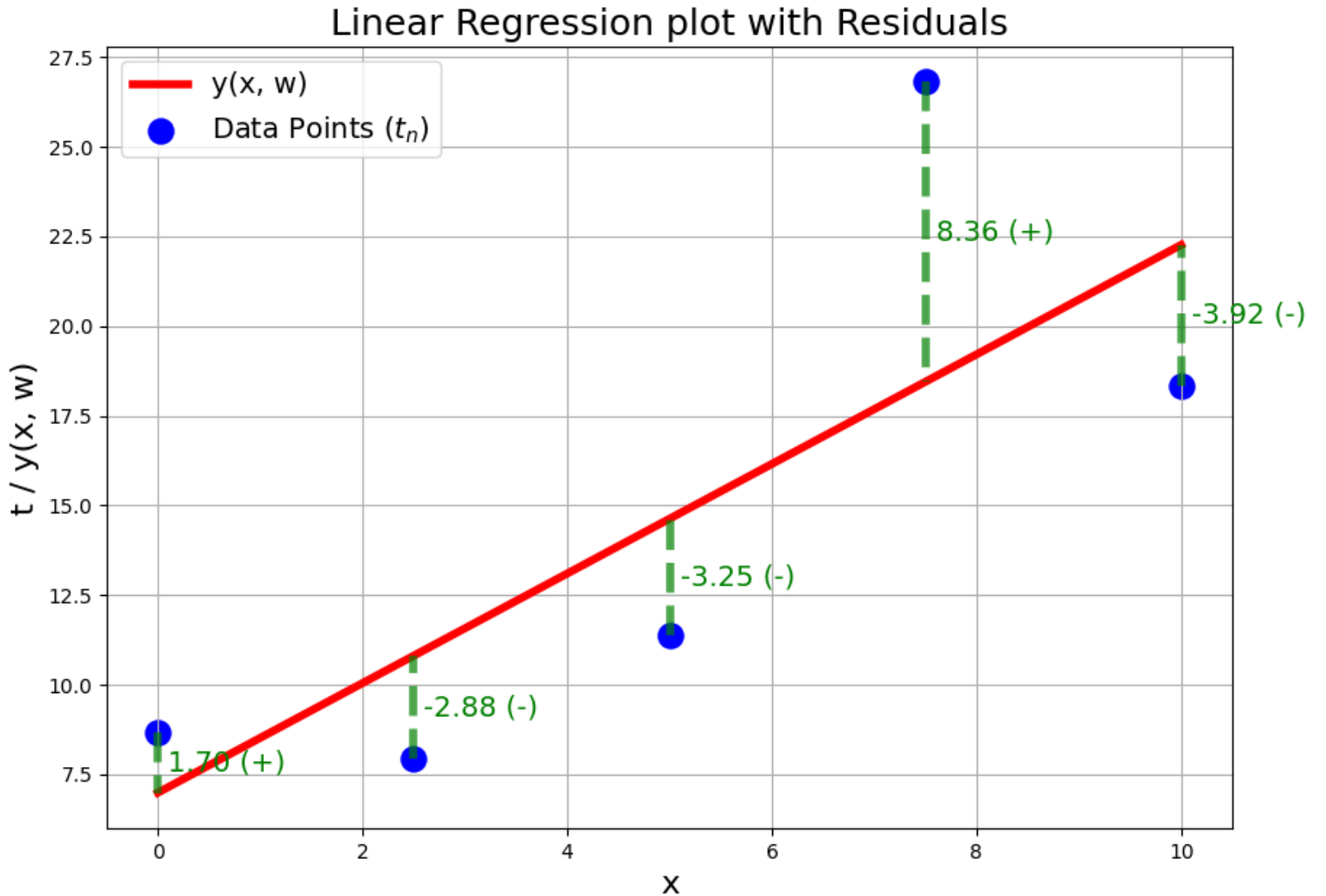
- Non-negative quantity.
- Only 0 if all predictions are equal to targets.

The factor of $\frac{1}{2}$ is included for later convenience.

Error Function Visualization

Residuals are defined as
 $e_n = (t_n - y(x_n, w))$
and they can be + or -.

The error function $E(w)$
combines the squares of
the residuals to measure
the goodness of the fit.



Error Function Minimization

- Linear Regression
- Basis Functions
- Vectorizing Calculations
- Error Function
- Error Function Minimization
- Geometric Interpretation
- Evaluation

Error Function Minimization

- $E(\mathbf{w})$ is a quadratic function.

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (\mathbf{t}_n - y(\mathbf{x}_n, \mathbf{w}))^2$$

Therefore, $\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}}$ is a linear function of $\mathbf{w}$ and hence solving $\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = 0$ has one and only one solution. Let's now derive this solution:

$$= \frac{1}{2} \sum_{n=1}^N \left(\mathbf{t}_n - \mathbf{w}_0 - \sum_{j=1}^D \mathbf{w}_j \mathbf{x}_{nj} \right)^2 \quad \xrightarrow{\text{Finding the optimum solution}} \quad \frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = 0$$

Error Function Minimization

$$\frac{\partial E(\mathbf{w})}{\partial w_0} = 0 \longrightarrow \frac{\partial \left(\frac{1}{2} \sum_{n=1}^N \left((t_n - w_0 - \sum_{d=1}^D w_d x_{nd}) \right)^2 \right)}{\partial w_0} = 0$$

Sum rule

$$\frac{\partial \sum f}{\partial w_0} = \sum \frac{\partial f}{\partial w_0} = \frac{1}{2} \sum_{n=1}^N \frac{\partial \left((t_n - w_0 - \sum_{d=1}^D w_d x_{nd})^2 \right)}{\partial w_0}$$

Chain rule

$$\frac{\partial f^2}{\partial f} = 2f \frac{\partial f}{\partial w_0} \implies = \frac{1}{2} \sum_{n=1}^N 2 \left(t_n - w_0 - \sum_{d=1}^D w_d x_{nd} \right) \left(\frac{\partial (t_n - w_0 - \sum_{d=1}^D w_d x_{nd})}{\partial w_0} \right)$$

Separating the terms

$$= \sum_{n=1}^N \left(t_n - w_0 - \sum_{d=1}^D w_d x_{nd} \right) (-1) = - \left(\sum_{n=1}^N t_n - N w_0 - \sum_{n=1}^N \left(\sum_{d=1}^D w_d x_{nd} \right) \right)$$

$$\frac{\partial E(\mathbf{w})}{\partial w_0} = 0 \longrightarrow w_0 = \frac{1}{N} \left(\sum_{n=1}^N t_n - \sum_{n=1}^N \left(\sum_{d=1}^D w_d x_{nd} \right) \right)$$

Error Function Minimization

$$\frac{\partial E(\mathbf{w})}{\partial w_k} = 0 \quad \longrightarrow \quad \frac{\partial \left(\frac{1}{2} \sum_{n=1}^N \left((t_n - w_0 - \sum_{d=1}^D w_d x_{nd}) \right)^2 \right)}{\partial w_k} = 0$$

Sum rule

$$\frac{\partial \sum f}{\partial w_k} = \sum \frac{\partial f}{\partial w_k} = \frac{1}{2} \sum_{n=1}^N \frac{\partial \left((t_n - w_0 - \sum_{d=1}^D w_d x_{nd})^2 \right)}{\partial w_k}$$

Chain rule

$$\frac{\partial f^2}{\partial f} = 2f \frac{\partial f}{\partial w_k} \quad = \frac{1}{2} \sum_{n=1}^N 2 \left(t_n - w_0 - \sum_{d=1}^D w_d x_{nd} \right) \left(\frac{\partial (t_n - w_0 - \sum_{d=1}^D w_d x_{nd})}{\partial w_k} \right)$$

Reordering

$$\frac{\partial E(\mathbf{w})}{\partial w_k} = - \sum_{n=1}^N x_{nk} \left(t_n - w_0 - \sum_{d=1}^D w_d x_{nd} \right)$$

Error Function Minimization

$$\frac{\partial E(\mathbf{w})}{\partial w_k} = - \sum_{n=1}^N x_{nk} \left(t_n - w_0 - \sum_{d=1}^D w_d x_{nd} \right) = 0$$

$$\sum_{n=1}^N x_{nk} t_n = \sum_{n=1}^N x_{nk} \left(w_0 + \sum_{d=1}^D w_d x_{nd} \right) \rightarrow \underbrace{\sum_{n=1}^N x_{nk} t_n}_{\mathbb{X}^T T} = \underbrace{w_0 \sum_{n=1}^N x_{nk} + \sum_{d=1}^D w_d \sum_{n=1}^N x_{nk} x_{nd}}_{\mathbb{X}^T \mathbb{X} w}$$

Normal equations (正规方程) for the least squares problem

$$\mathbb{X}^T T = \mathbb{X}^T \mathbb{X} w$$

Multiplying both sides by $(\mathbb{X}^T \mathbb{X})^{-1}$

$$(\mathbb{X}^T \mathbb{X})^{-1} \mathbb{X}^T T = w$$

Takeaway $\rightarrow w^* = (\mathbb{X}^T \mathbb{X})^{-1} \mathbb{X}^T T$

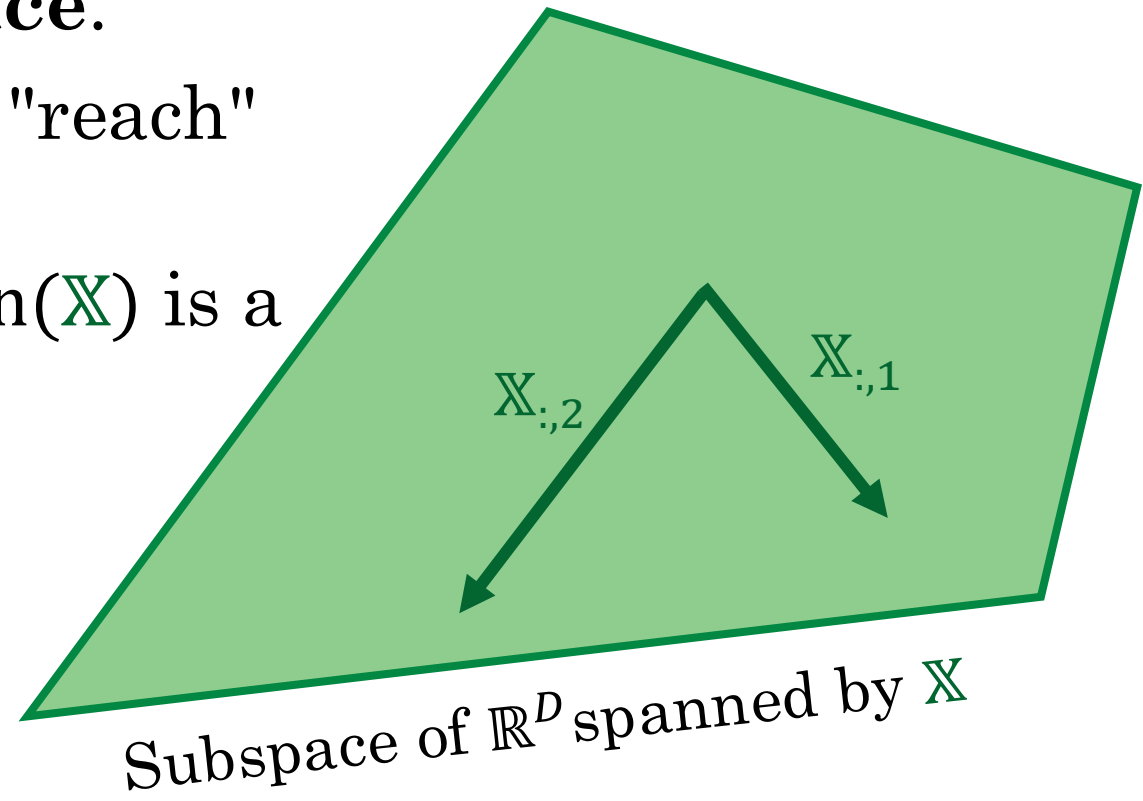
Assuming $\mathbb{X}^T \mathbb{X}$ is invertible

Geometric Interpretation

- Linear Regression
- Basis Functions
- Vectorizing Calculations
- Error Function
- Error Function Minimization
- Geometric Interpretation
- Evaluation

[Linear Algebra] Span (张成空间)

- The set of all possible **linear combinations of the columns** of $\mathbb{X}$ is called the **span** of $\mathbb{X}$ (denoted $\text{span}(\mathbb{X})$), also called the **column space**.
- Intuitively, this is all vectors you can "reach" using the columns of $\mathbb{X}$.
- If each column of $\mathbb{X}$ has length D , $\text{span}(\mathbb{X})$ is a subspace of $\mathbb{R}^D$.



[Linear Algebra] Matrix-Vector Multiplication

- **Approach 1:** So far, we've thought of our model as horizontally stacked predictions per data point:

$$\begin{bmatrix} | \\ \mathbb{Y} \\ | \end{bmatrix} = \begin{bmatrix} \text{---} & \mathbf{x}_1^T & \text{---} \\ \text{---} & \mathbf{x}_2^T & \text{---} \\ & \vdots & \\ \text{---} & \mathbf{x}_N^T & \text{---} \end{bmatrix} \begin{bmatrix} \text{---} \\ \text{---} \\ \text{---} \\ \text{---} \end{bmatrix} \begin{matrix} \\ \\ \\ \textcolor{violet}{w} \end{matrix}$$

Matrix sizes
 $N \times 1 = (N \times (D + 1)) ((D + 1) \times 1)$

- **Approach 2:** However, it is helpful sometimes to think of matrix-vector multiplication as performed by columns. We can also think of $\mathbb{Y}$ as a **linear combination of feature vectors**, scaled by **parameters**.

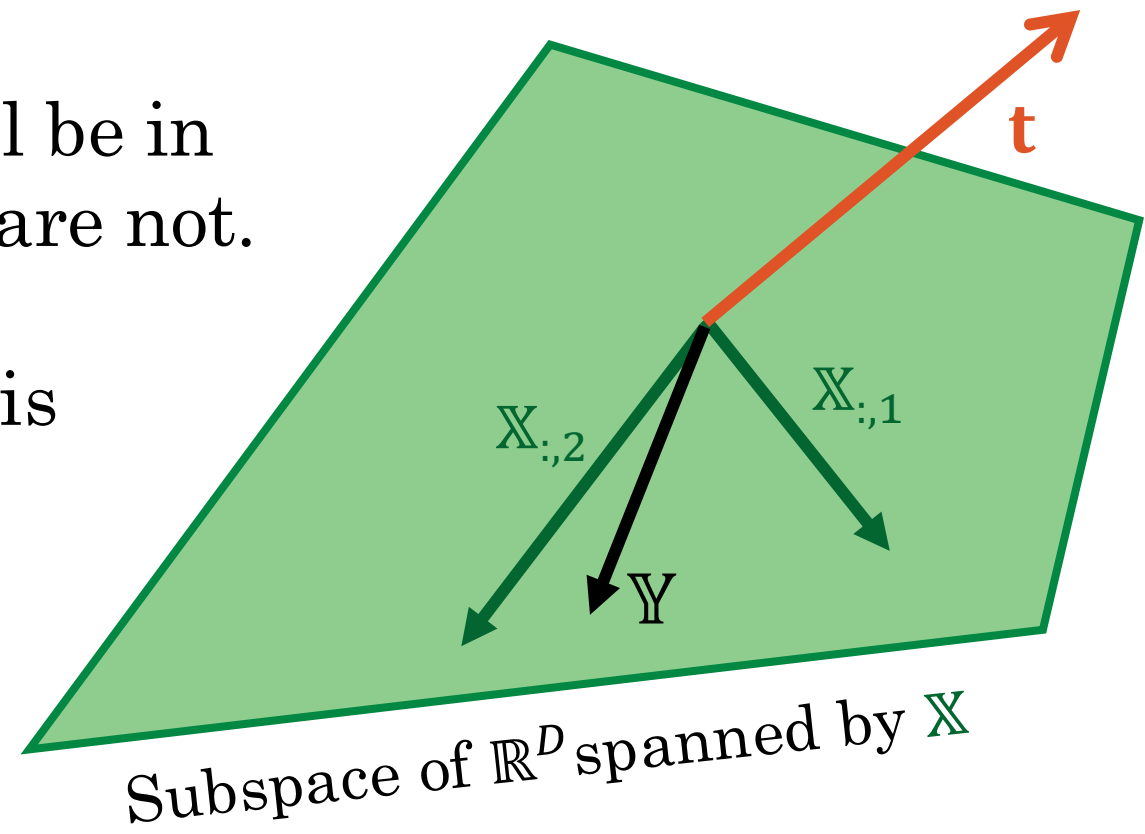
$$\begin{bmatrix} | \\ \mathbb{Y} \\ | \end{bmatrix} = \begin{bmatrix} | & & | \\ \textcolor{teal}{\mathbb{X}}_{:,0} & \dots & \textcolor{teal}{\mathbb{X}}_{:,D} \\ | & & | \end{bmatrix} \begin{bmatrix} \text{---} \\ \text{---} \\ \text{---} \\ \text{---} \end{bmatrix} \begin{matrix} \\ \\ \\ \textcolor{violet}{w} \end{matrix} = \textcolor{violet}{w}_0 \begin{bmatrix} | \\ \textcolor{teal}{\mathbb{X}}_{:,0} \\ | \end{bmatrix} + \textcolor{violet}{w}_1 \begin{bmatrix} | \\ \textcolor{teal}{\mathbb{X}}_{:,1} \\ | \end{bmatrix} + \dots + \textcolor{violet}{w}_D \begin{bmatrix} | \\ \textcolor{teal}{\mathbb{X}}_{:,D} \\ | \end{bmatrix}$$

Prediction Is a Linear Combination of Columns

Our prediction of $\mathbb{Y}(\mathbb{X}, \vec{w}) = \mathbb{X} \vec{w}$ is a linear combination of columns of $\mathbb{X}$.

Interpret: Our linear prediction $\mathbb{Y}$ will be in $\text{span}(\mathbb{X})$, even if ground-truth values $\mathbf{t}$ are not.

Goal: Find vector of $\mathbb{Y}$ in $\text{span}(\mathbb{X})$ that is closest to $\mathbf{t}$.



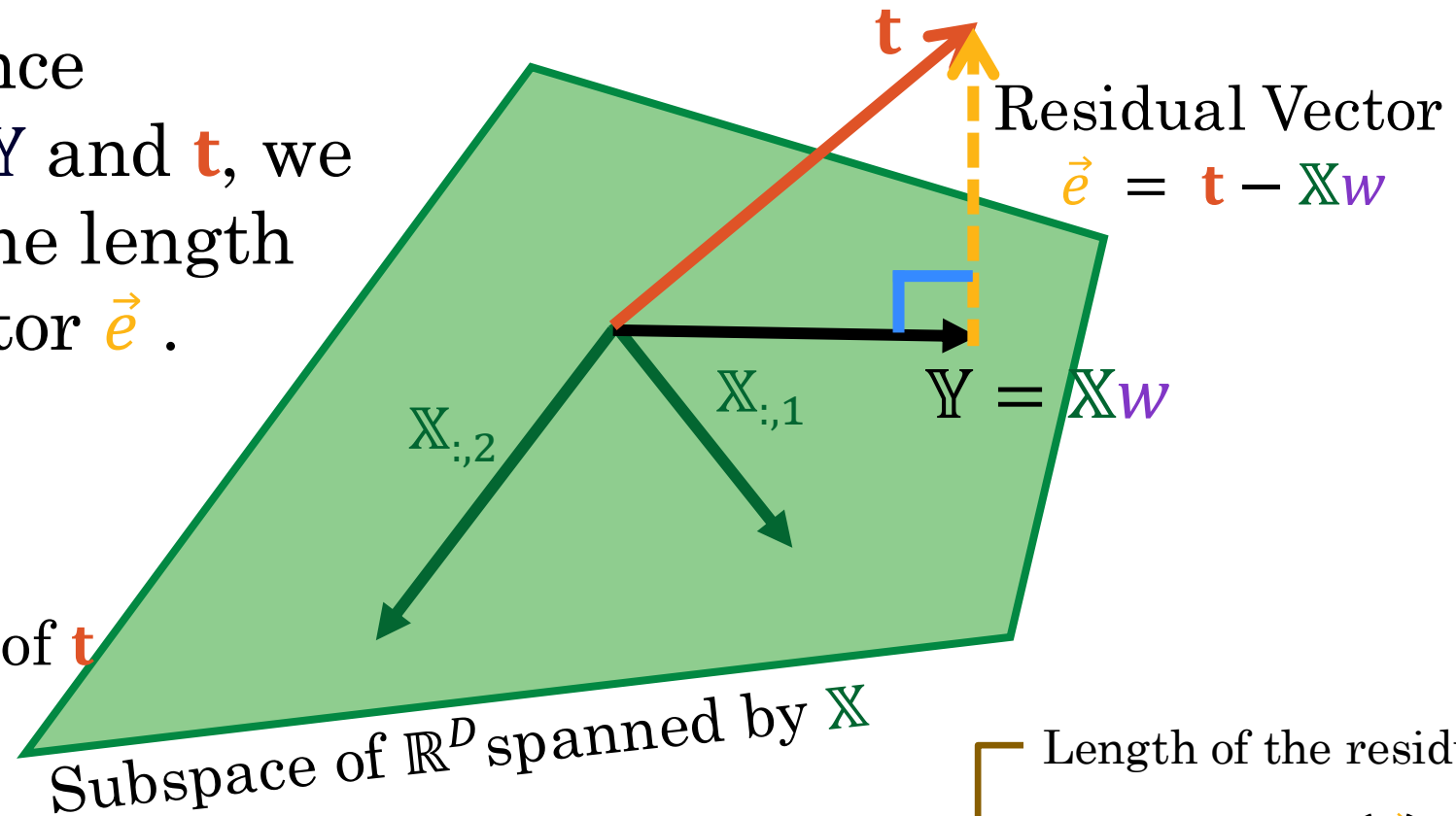


How to find ‘closest point in a subspace’?

Finding Optimum Predictions ($\mathbf{Y}$)

To minimize distance between vector of $\mathbf{Y}$ and $\mathbf{t}$, we should minimize the length of the residual vector $\vec{e}$.

$\vec{e}$ is minimized if the prediction $\mathbf{Y}$ is the orthogonal projection of $\mathbf{t}$ on the $\text{span}(\mathbf{X})$.

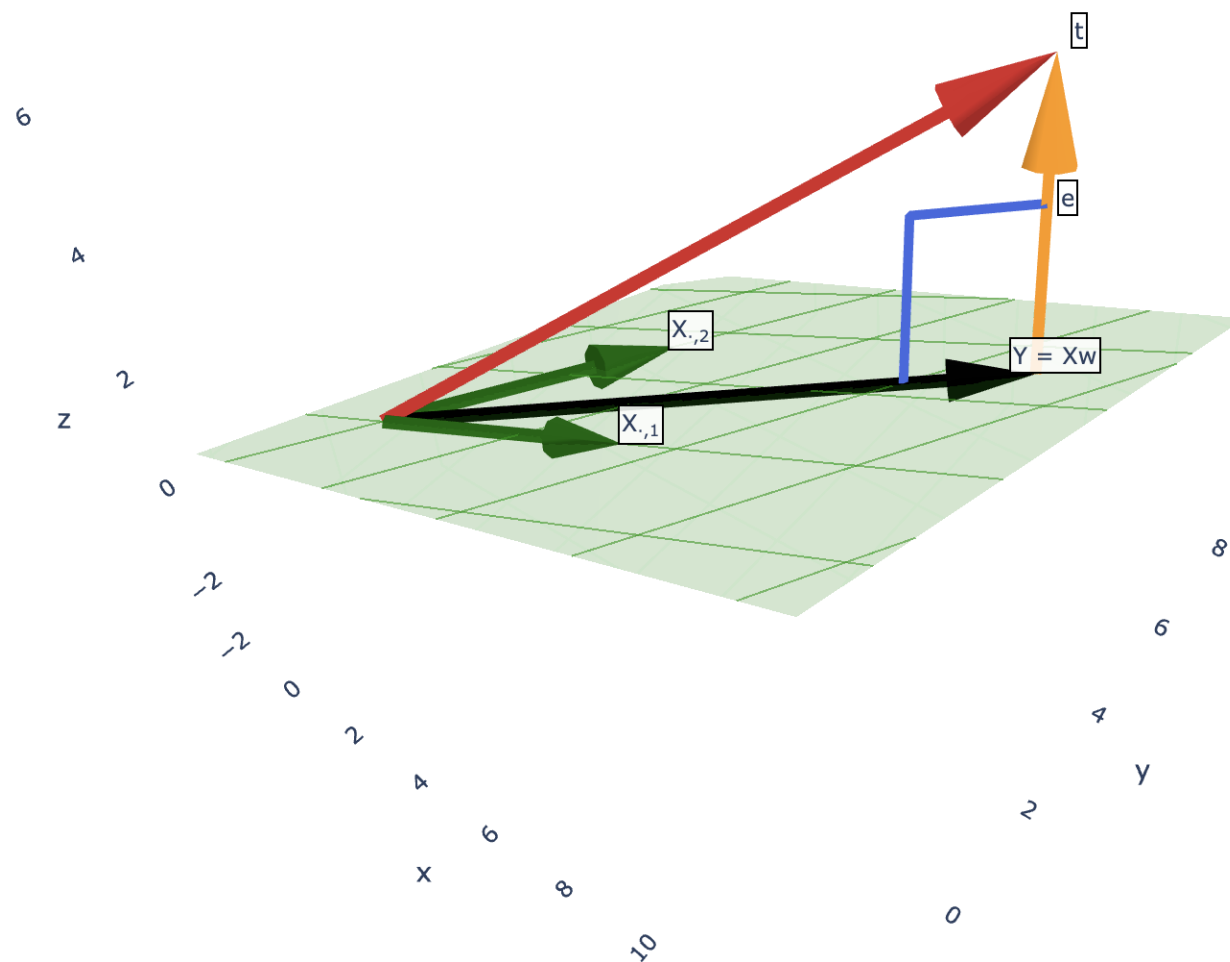


Length of the residual vector

$$L_2 - \text{norm}(\vec{e}) = \|\vec{e}\|_2$$

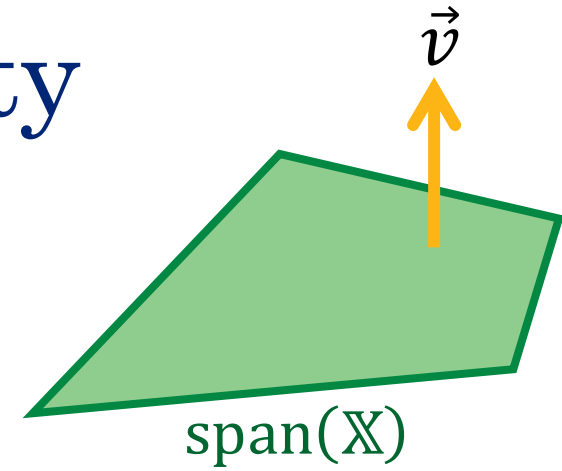
$$= \sqrt{\sum_{d=0}^D e_d^2}$$

Geometry of Least Squares in Plotly



[Linear Algebra] Orthogonality

- A vector $\vec{v}$ is orthogonal to the $\text{span}(\mathbb{X})$ if and only if it is orthogonal to all columns of $\mathbb{X}$.



Assuming $\vec{v} \in \mathbb{R}^N$ and $\mathbb{X} = \begin{bmatrix} | & & | \\ \mathbb{X}_{:,0} & \dots & \mathbb{X}_{:,D} \\ | & & | \end{bmatrix} \in \mathbb{R}^{N \times D}$

$\vec{v}$ orthogonal to each column means:

$$\mathbb{X}_{:,0}^T \vec{v} = 0$$

$$\mathbb{X}_{:,1}^T \vec{v} = 0$$

$$\vdots$$

$$\mathbb{X}_{:,D}^T \vec{v} = 0$$

$$\Rightarrow \begin{bmatrix} \mathbb{X}_{:,0}^T \vec{v} \\ \mathbb{X}_{:,1}^T \vec{v} \\ \vdots \\ \mathbb{X}_{:,D}^T \vec{v} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$\Rightarrow \mathbb{X}^T \vec{v} = 0$$

We will use
this shortly

Going Back to Our Error Function

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (\mathbf{t}_n - y(\mathbf{x}_n, \mathbf{w}))^2 = \frac{1}{2} (\|\vec{e}\|_2)^2$$

Equivalently, vector $\vec{e}$ is orthogonal to the $\text{span}(\mathbb{X})$, meaning that:

$$\mathbb{X}^T \vec{e} = 0 \quad \text{Adding the definition of residual } \vec{e} = \mathbf{t} - \mathbb{Y}(\mathbb{X}, \mathbf{w}^*)$$

$$\mathbb{X}^T (\mathbf{t} - \mathbb{Y}(\mathbb{X}, \mathbf{w}^*)) = 0 \quad \text{Adding formula for } \mathbb{Y}(\mathbb{X}, \mathbf{w}^*) = \mathbb{X} \mathbf{w}^*$$

$$\mathbb{X}^T \mathbf{t} - \mathbb{X}^T \mathbb{X} \mathbf{w}^* = 0 \quad \text{Moving terms}$$

$$\mathbb{X}^T \mathbf{t} = \mathbb{X}^T \mathbb{X} \mathbf{w}^*$$

Normal
Equation

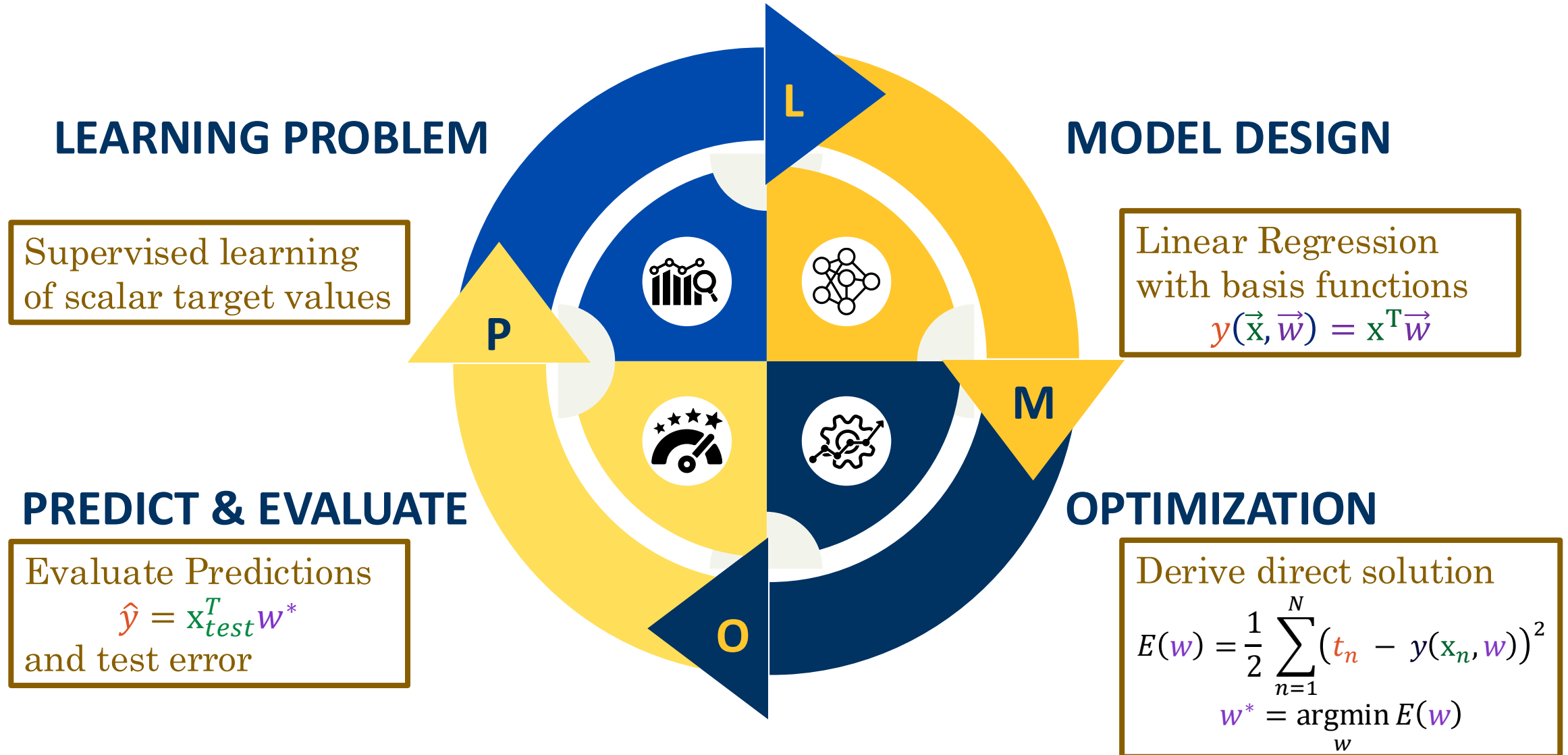
If $\mathbb{X}^T \mathbb{X}$ is
invertible

$$\mathbf{w}^* = (\mathbb{X}^T \mathbb{X})^{-1} \mathbb{X}^T \mathbf{t}$$

Evaluation

- Linear Regression
- Basis Functions
- Vectorizing Calculations
- Error Function
- Error Function Minimization
- Geometric Interpretation
- Evaluation

Predict and Evaluate



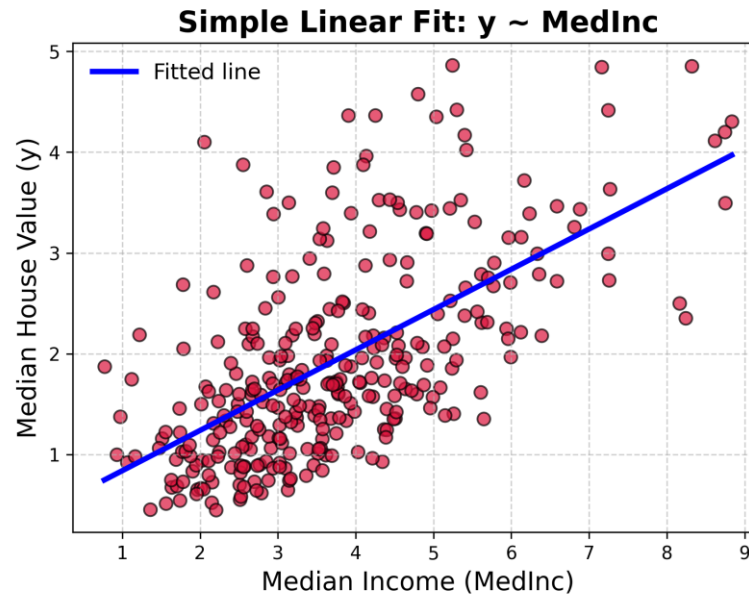
Evaluation - Visualization

Residual plot shows the trend of the residuals $e_n = (t_n - y(x_n, w))$ with respect to predictions $y(x_n, w)$.

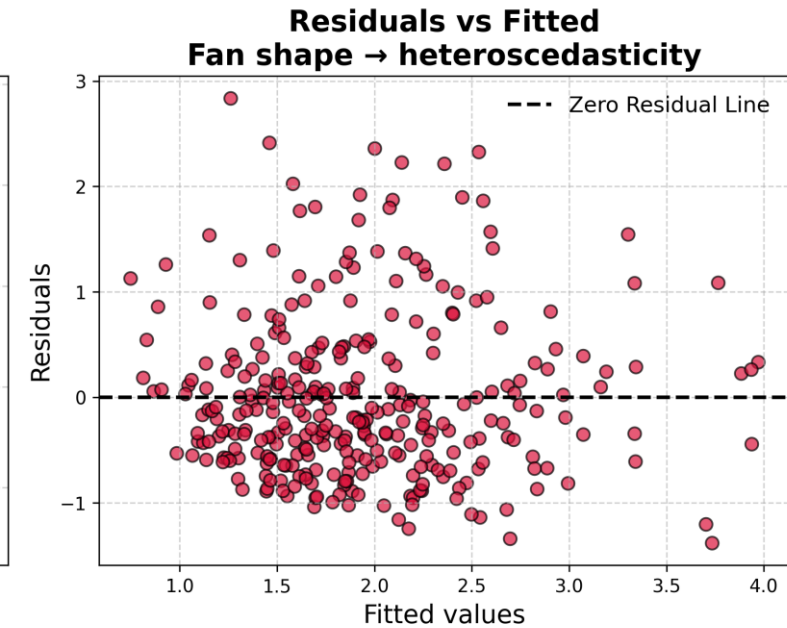
What a good residual plot looks like:

- Points scattered randomly around 0 (no pattern).
- Roughly constant vertical spread (homoscedasticity).
- No obvious trend with fitted values or with predictors.

Fitting Plot



Residual Plot



Evaluation - Metrics

Evaluation - Metrics

Mean Squared Error (MSE)

$$\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)$$

Evaluation - Metrics

Mean Squared Error (MSE)

$$\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)$$

Root Mean Squared Error (RMSE)

$$\sqrt{\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)}$$

Moves the metric back to the original unit of the data compared to MSE

Evaluation - Metrics

Mean Squared Error (MSE)

$$\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)$$

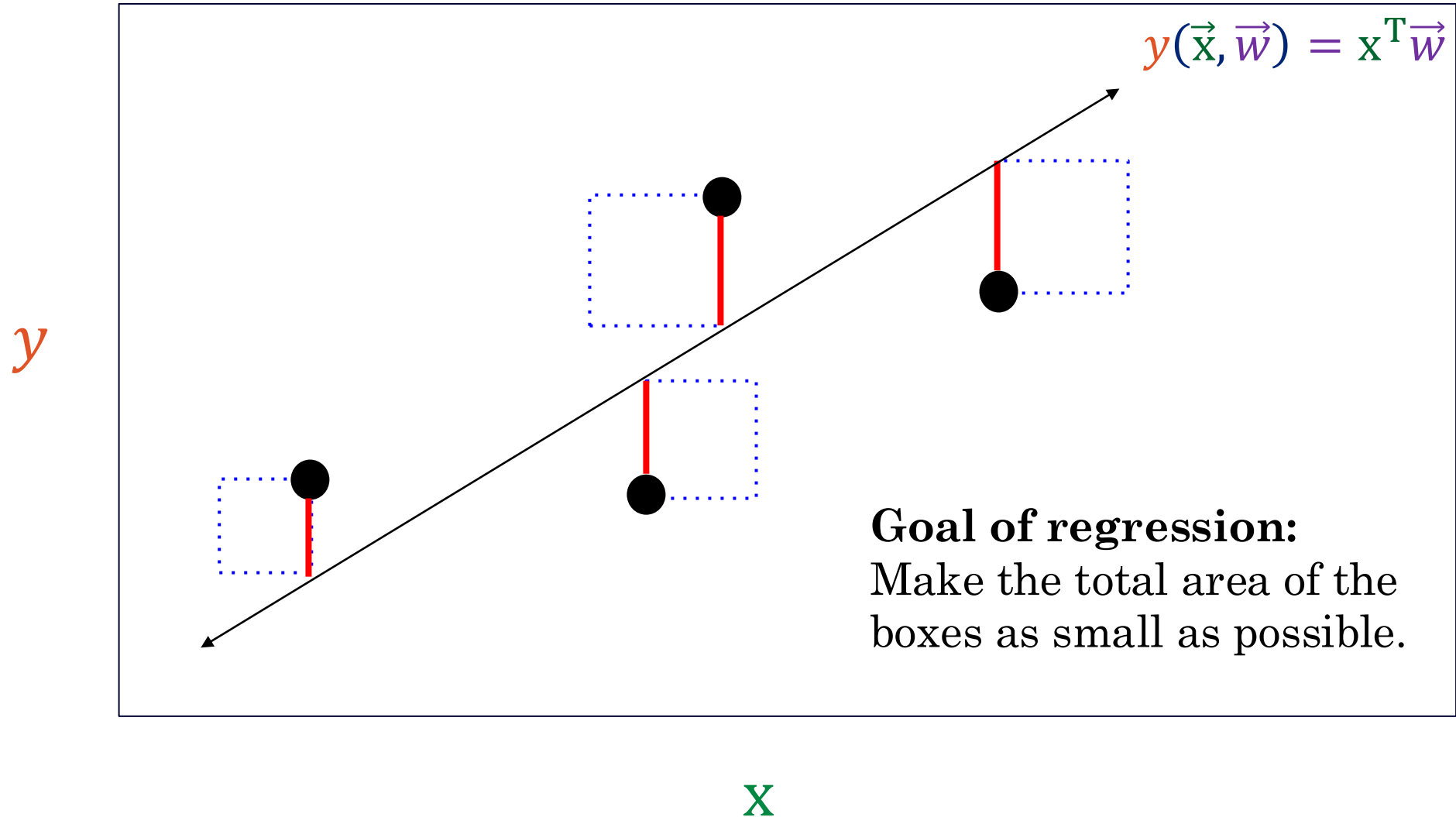
Root Mean Squared Error
(RMSE)

$$\sqrt{\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)}$$

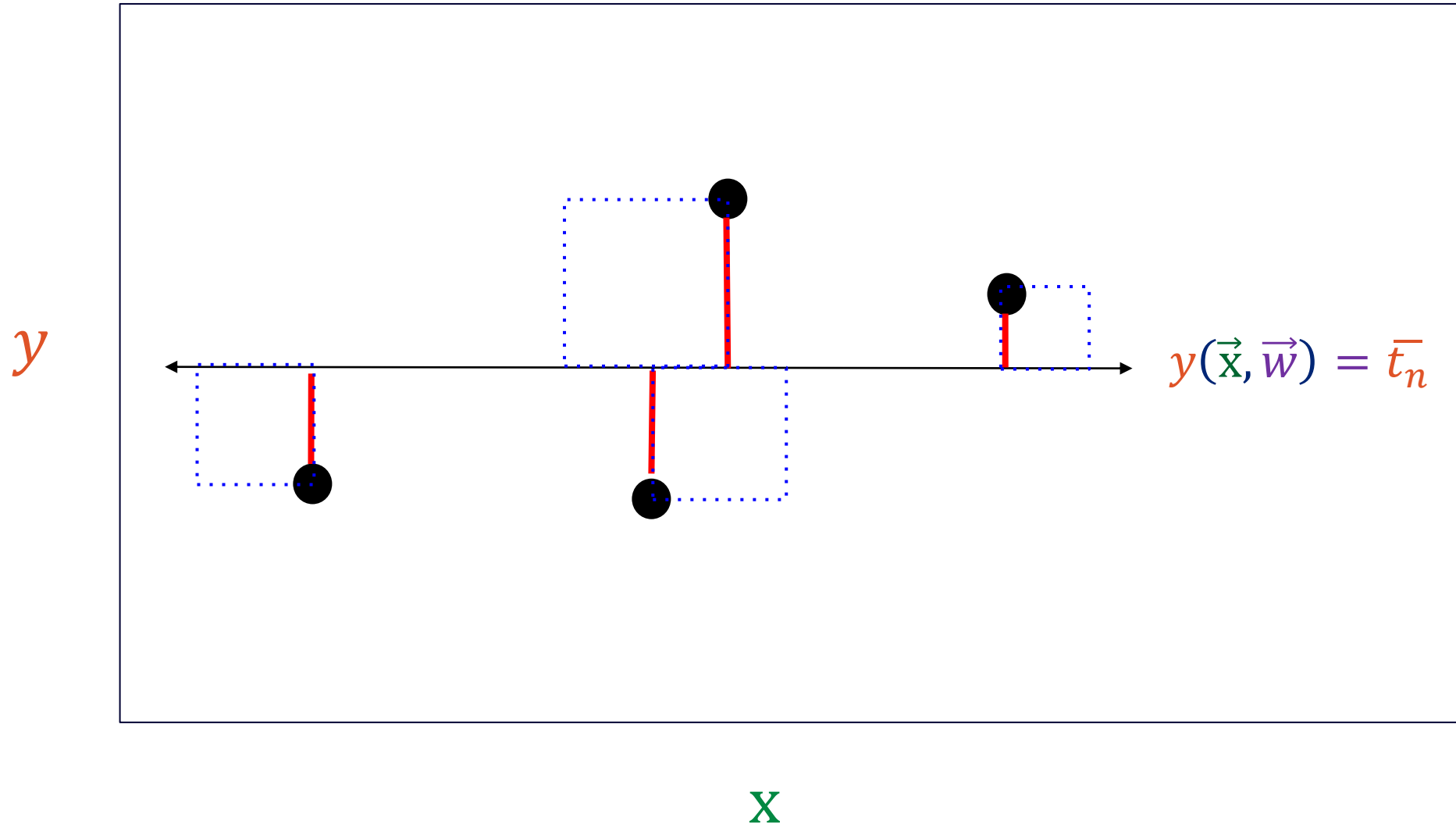
R-Squared (R^2) Score

$$1 - \frac{\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2}{\sum_{n=1}^N (t_n - \bar{t})^2}$$

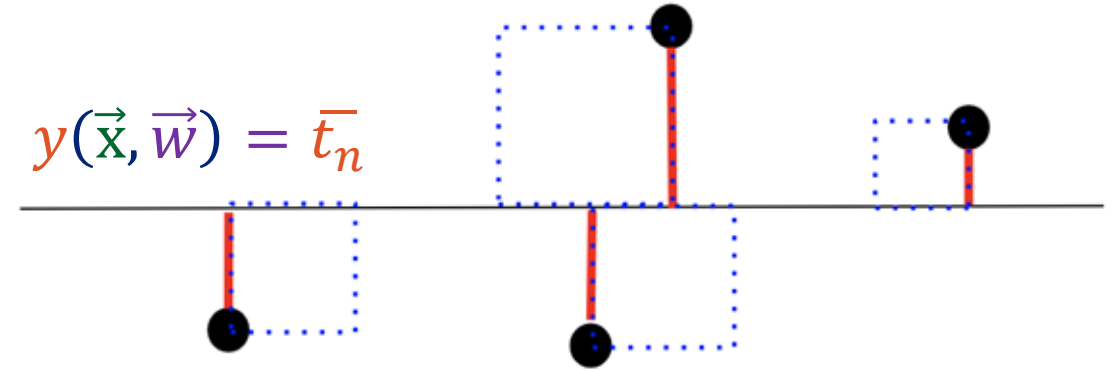
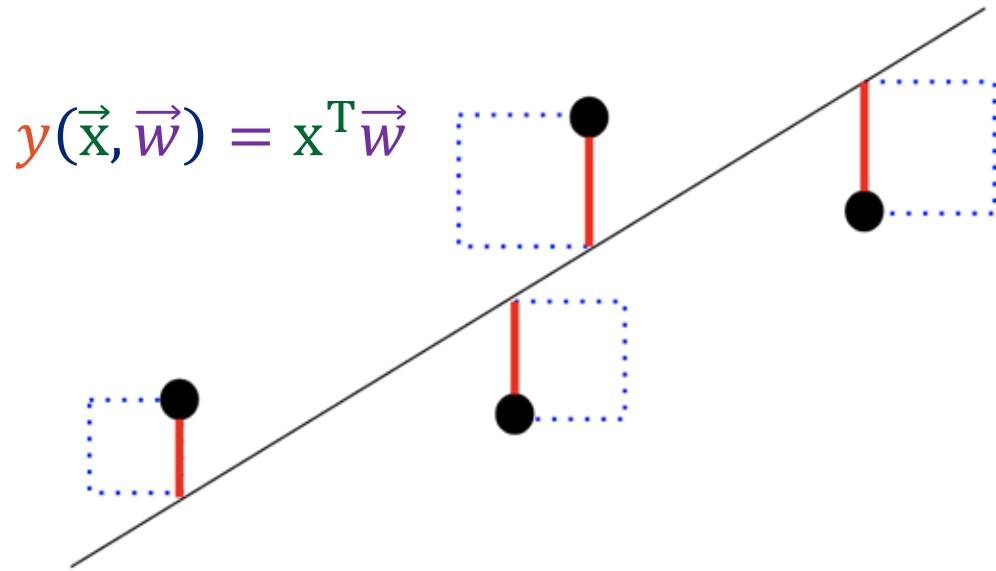
Visualizing the Sum of Squared Error of Regression Model



Visualizing the Sum of Squared Error of Intercept Model



R²: Quality of the Fit Relative to Intercept Model



$$R^2 = \frac{\Delta \text{ in area}}{\text{Constant Model Area}} = 1 - \frac{\sum_{n=1}^N (t_n - y(\vec{x}_n, \vec{w}))^2}{\sum_{n=1}^N (t_n - \bar{t}_n)^2}$$

unitless and only compares performance relative to mean baseline.

Evaluation - Metrics

Mean Squared Error (MSE)

$$\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)$$

Root Mean Squared Error (RMSE)

$$\sqrt{\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)}$$

R-Squared (R^2) Score

$$1 - \frac{\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2}{\sum_{n=1}^N (t_n - \bar{t})^2}$$

Mean Absolute Error (MAE)

$$\frac{1}{N} \left(\sum_{n=1}^N |t_n - y(\mathbf{x}_n, \mathbf{w})| \right)$$

In the same unit as the data; similar to MSE but differs in how the penalization applies.

Evaluation - Metrics

Mean Squared Error (MSE)

$$\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(x_n, w))^2 \right)$$

指标	单位 (Unit)	惩罚机制 (Penalization)	对离群点 (Outliers)
MAE	原始单位	线性：误差多大，惩罚多大	不敏感 (Robust)
MSE	原始单位的平方	二次方：误差越大，惩罚指数级增加	极度敏感

Mean Absolute Error (MAE)

$$\frac{1}{N} \left(\sum_{n=1}^N |t_n - y(x_n, w)| \right)$$

In the same unit as the data; similar to MSE but differs in how the penalization applies.

Evaluation - Metrics

Mean Squared Error (MSE)

$$\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)$$

Root Mean Squared Error (RMSE)

$$\sqrt{\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)}$$

R-Squared (R^2) Score

$$1 - \frac{\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2}{\sum_{n=1}^N (t_n - \bar{t})^2}$$

Mean Absolute Error (MAE)

$$\frac{1}{N} \left(\sum_{n=1}^N |t_n - y(\mathbf{x}_n, \mathbf{w})| \right)$$

Mean Absolute Percentage Error (MAPE)

$$\frac{1}{N} \left(\sum_{n=1}^N \left| \frac{t_n - y(\mathbf{x}_n, \mathbf{w})}{t_n} \right| \right) \times 100$$

Lecture 5

Linear Regression (1)

Reference Book Chapters: Chapter 1.[2.1-2.3], Chapter 4.[1.1, 1.4]